L14: Deep Embeddings and Graph Neural Networks

Deep Learning (i.e., the use of neural networks with several layers of hidden units) gives us models that can learn quite complex nonlinear and hierarchical input-output functions.

The node embedding methods we discussed so far are "shallow" in the sense that they are not computed using such deep neural networks. There are several problems with shallow embeddings:

- The model parameters include **a different embedding vector for each node**. For huge graphs, this can be an issue. Ideally, we would like to compute the embedding vector of any node using a single model, sharing the model parameters across all nodes.
- Models based on shallow embeddings **do not have inductive capability,** i.e., they cannot generalize beyond the training data that we use to learn the model. An embedding vector is computed for every node of the given graph -- but it is not clear how to compute embedding vectors for nodes that may join the graph later (think of dynamic graphs) or for nodes that are not visible (we may only have a partial view of the compete graph). Ideally, we would like to learn a model that can generalize beyond the portion of the graph it has used in the training process.
- The shallow embedding methods we have discussed are based strictly on connectivity (i.e., the graph adjacency matrix) and so they **cannot model arbitrary node attributes**. For instance, in a social network the nodes may have additional attributes that relate to their gender, age, salary, etc.

We will now present one way to apply deep learning in the problem of node embedding and graph representation: **Graph Neural Networks (or GNNs).** We emphasize that there are many other similar methods, such as Graph Convolutional Networks or Graph Recurrent Neural Networks. If you are interested to learn more, please refer to the reading list at the start of the Lesson.

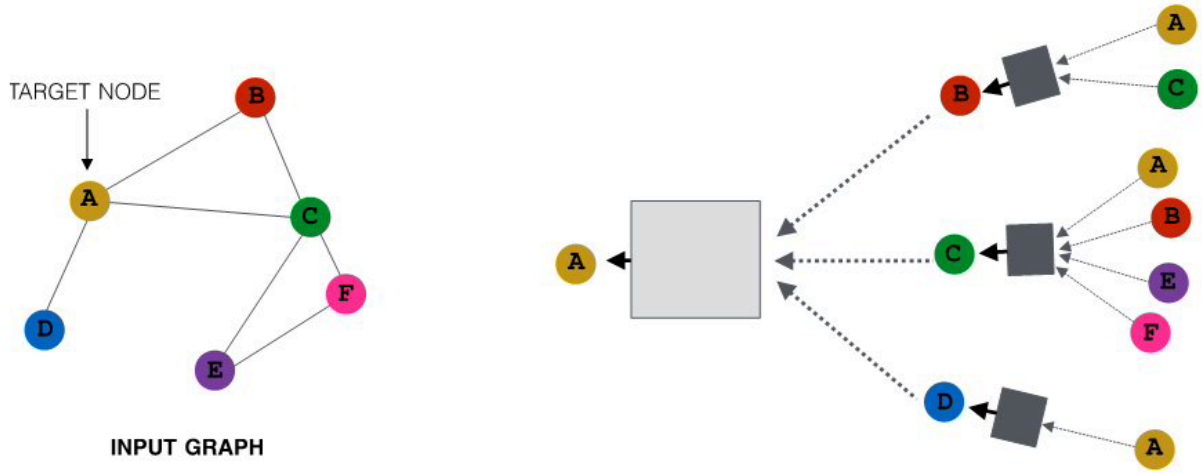
Suppose that we are given a graph G with:

- ***V***: the set of ***n*** nodes.
- ***A***: the adjacency matrix.
- $\boldsymbol{x_v}$: an m-dimensional vector for each node **v,** representing arbitrary node attributes (e.g., gender, age, salary)

A GNN is a neural network in which *we compute an embedding vector for each node at each layer of the network.* Let us represent by $\boldsymbol{h_v^k}$ the embedding of node ***v*** at layer ***k***. At the input layer, we have that $\boldsymbol{h_v^0 = x_v}$, i.e., the given node attributes.

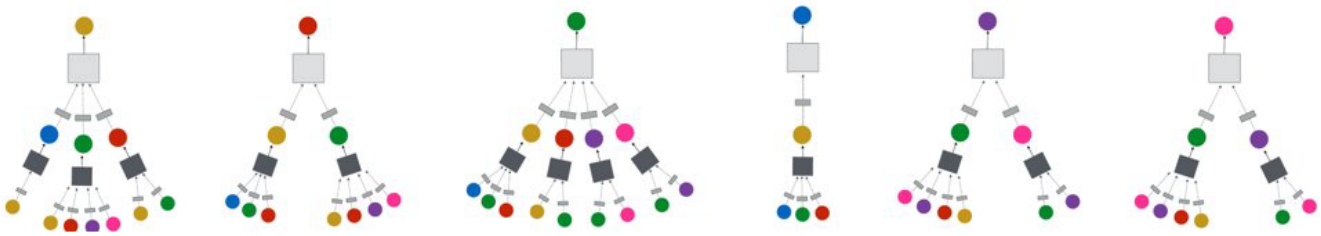
At the ***k'th*** hidden layer (***k>0***), the embedding $\boldsymbol{h_v^k}$ of node ***v*** depends on both the embedding of the same node at the previous layer $\boldsymbol{h_v^{k-1}}$ as well as on the embedding $\boldsymbol{h_u^{k-1}}$ of every neighbor ***u*** of ***v*** at the previous layer. More specifically, if ***N(v)*** represents the set of all neighbors of node v in the given graph, the embedding $\boldsymbol{h_v^k}$ depends on the ***average*** of $\boldsymbol{h_u^{k-1}}$ across all ***u*** in ***N(v)***. This average represents the contribution of the "local neighborhood" of node v to its embedding.

For example, consider the graph at the left of the following figure. The neural network at the right shows that the embedding of node-A at the second hidden layer depends on the layer-1 embeddings of nodes B, C and D (the neighbors of A). Similarly, the embedding of node B at layer-1 depends on the layer-0 embeddings of nodes A and C (the neighbors of B), the embedding of node C at layer-1 depends on the layer-0 embedding of nodes A, B, E and F, and the embedding of node D at layer-1 depends on the layer-0 embedding of node A.



All images in this page are from Jure Leskovec , “Representation Learning on Networks” tutorial, WWW 2018

The previous figure only showed the neural network for computing the embedding of node-A (the yellow node) at layer-2. The following figure shows the corresponding networks for all other nodes, using the same color code. Of course in practice all these networks would be integrated in the same neural network model.



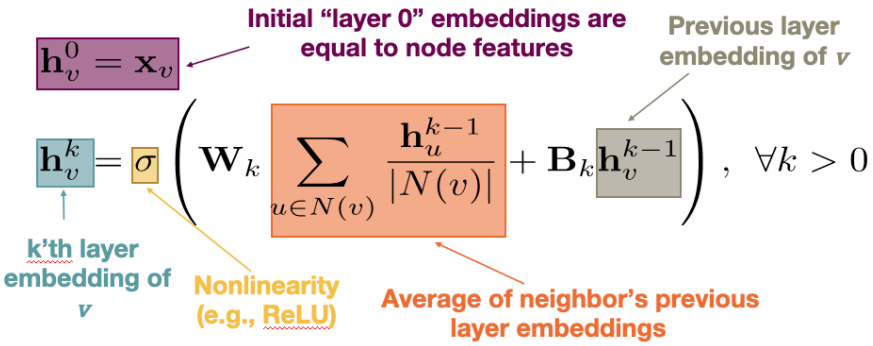
What is the specific mathematical form of these functional dependencies? As in most artificial neural networks, the output of a hidden-layer unit with inputs $\boldsymbol{x}$ (a $\boldsymbol{d}$-dimensional vector) is given by a nonlinear *activation function* $\sigma()$ of the weighted sum of the inputs:

$$\sigma\left(\sum_{i=0}^{d} w_i x_i\right)$$

where $\boldsymbol{w}$ is the vector of model parameters (the weight of each input) that we will learn in the training process. The model potentially includes a "bias" term $\boldsymbol{w_0}$ for $\boldsymbol{x_0}=1$. The nonlinear function $\sigma()$ could be a **sigmoid** (https://en.wikipedia.org/wiki/Sigmoid_function) function between 0 and 1 -- but recently most models use the **ReLU** (https://en.wikipedia.org/wiki/Rectifier_(neural_networks)) function, which is simpler to compute.

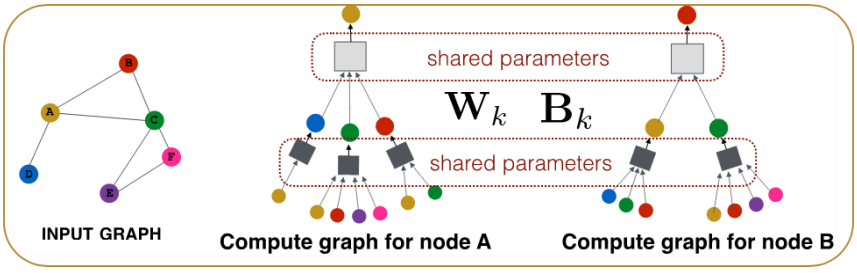
The input weights at layer-k are represented by the matrices $\boldsymbol{W_k}$ (applied on the average neighbor embedding from the previous layer) and $\boldsymbol{B_k}$ (applied on the embedding of the node at the previous layer). We will see how to compute these matrices in the next page, when we discuss how to train GNN models.

We are now ready to present the complete equation for the embedding of node $\boldsymbol{v}$ at layer $\boldsymbol{k}$ -- *this is exactly what the GNN computes for each node and at each layer:*



Note that the model captures the topology of the graph through the local neighborhood ***N(v)*** of each node. At the first hidden layer, the model only "knows" about the direct neighbors of each node. At layer-2, however, it also knows about the neighbors of the neighbors. If the neural network is sufficiently deep (large ***k***), the model can learn quite subtle structural properties of the graph because it has information about all neighbors of each node within a broader neighborhood that is ***k***-hop wide.

Another important point in the previous equation is that the parameters of the model, given by the matrices $\boldsymbol{W_k}$ and $\boldsymbol{B_k}$ for each layer, are shared across nodes at layer-***k***, i.e., *we do not need to learn different parameters for each node.* This is how GNNs accomplish two important goals: first, significant reduction in the complexity of the model (because they do not need to learn different parameters for each node) and second, the model has inductive capabilities because it can apply these matrices even on nodes that are not in the training dataset.



Food For Thought

a) There is a "hidden assumption" behind the idea of using the same parameters for all nodes at each layer. How would you precisely state that assumption?

b) What is the rationale for NOT using the same shared parameters at every layer?